Welcome to your next lesson: Meet the while loop.
I'm preparing your personalized lesson based on what you've learned so far, what's coming up next, and how to help you build on your skills.
Step by step, you're moving forward.

- Imagine you're making a little game ?
  You start with some number of lives, but you don't know that number when you write the code. The rule is:
  > "*Keep playing while you still have at least 1 life.*"

  You can't easily use a `for` loop here, because `for` is best when you already know how many times to repeat (like "do this 10 times").
  A **`while`** loop is different:
  > *It keeps repeating as long as a condition is True*
  > *(for example: **`while`** `lives > 0`).*

  We'll use it for "repeat until something changes" situations.
  Quick check, just your gut feeling:
  Does the game-lives rule sound like a good fit for a while loop, a for loop, or are you not sure yet?
  > I think it's a while loop

  Nice, that's exactly right ?
  A while loop is perfect for: "keep doing this until something changes."
  Very simple picture:

```Python
while condition_is_true:
    # repeat this block
```

  Each time Python:

1. Checks the condition
2. If it's **`True`** → runs the block
3. Then goes back to step 1 again

  Make sense so far: a **`while`** loop = "repeat while this condition is True"?
  Just answer: yes or not really and I'll adjust.
  > Yes

  Great ✅
  Now let's zoom in on when to choose a **`while`** loop instead of a `for` loop.
  Very roughly:
- Use `for` when you know the exact number of repeats (like "do this 5 times").

- Use while when you don't know how many times, but you know a condition that must stay True (like "while lives > 0", "while password is wrong", "while user is not done").

  Example idea (no need to code yet):

  Python
  ```
  lives = 3
  while lives > 0:
      print("You still have a life!")
      lives = lives - 1
  ```

  This will keep going while lives > 0 is True.

  Does that difference between for (known count) and while (condition) feel clear enough, or a bit fuzzy?

  You can say "clear" or "fuzzy".

  Clear

  Awesome ?

  Next, let's look at the three parts every while loop needs (this is how we avoid bugs):

1. Initialize a variable before the loop
2. Test a condition in the while line
3. Update something inside the loop so the condition will eventually become False

   Tiny example:

   Python
   ```
   counter = 1          # 1) initialize

   while counter <= 3:  # 2) test
       print(counter)
       counter = counter + 1   # 3) update
   ```

   This prints 1, 2, 3 and then stops because counter <= 3 becomes False.

   Does this 3-step pattern (initialize → test → update) make sense, or should I slow down on one of the steps?

   Makes sense

   Perfect ?

   Now let's look at what happens if we forget the update step inside a while loop.

   Example:

   Python
   ```
   counter = 1

   while counter <= 3:
       print(counter)
       # Oops! forgot: counter = counter + 1
   ```

Step by step:

- counter starts at 1
- while counter <= 3 → 1 <= 3 → True → prints 1
- Then it checks again… but counter is still 1
- Condition is still True → prints 1 again
- And again… forever ?

This is called an infinite loop: the condition never becomes False, so the loop never ends.

Does this “forgot the update → infinite loop” idea feel clear?

If yes, I’ll show you a small visual-style table of how counter changes in a correct loop.

Yes, it's clear

Great ?

Let’s walk through a correct while loop step by step so you can really see it run.

Code:

```python
counter = 1

while counter <= 3:
    print(counter)
    counter = counter + 1
```

Let’s trace it in a little table:

Loop check #	counter before check	Condition counter <= 3	Printed	counter after update
1	1	True	1	2
2	2	True	2	3
3	3	True	3	4
4	4	False → loop stops	—	—

Because we update counter, the condition eventually becomes False and the loop ends ✅

Does this step-by-step table match how you imagine the loop running?

Yes, it matches

Nice ?

One more big safety idea about while loops: infinite loops.

An infinite loop is a loop that never ends because its condition never becomes False.

This usually happens when:

- The while condition is always True, or
- You forgot to update the variable that the condition is checking.

Example bad idea:

```Python
while True:
    print("Still going...")
```

This will print forever and basically “lock” your program until you stop it.

So, rule of thumb:

Always plan: what will make this condition become False?

And make sure you update that thing inside the loop.

Ready to try writing a small **while** loop yourself in the code editor?

Just answer yes or not yet.

Yes

Nice ? Let’s do a tiny one.

Step by step, here’s the goal:

*In the code editor, write a **while** loop that prints the numbers 1, 2, 3, 4, 5 (each on its own line).*

Hints to guide you (follow these in order):

1. Before the loop, initialize a variable, for example `counter = 1`.
2. The **while** line should test that `counter` is still in range (up to 5).
3. Inside the loop:

- **print**(counter)
- then update it (add 1).

Please do this in the code editor (not in chat):

- Click </> Open code editor
- Type your code
- Hit Run to test
- When it works, hit Submit code

After you submit, I’ll check it and we’ll build one more useful **while** loop together ?

```Python
i = 1
while (i < 6):
    print(i)
    i = i + 1
```

Nice work, Ron ? Your loop is perfect:

```Python
i = 1
while (i < 6):
    print(i)
```

```
    i = i + 1
```

- Initialize: `i = 1`
- Test: `while (i < 6):`
- Update: `i = i + 1`

It stops exactly after printing 1 to 5 because `i` becomes 6, so `i < 6` is False.

Let's try a more "real world" style loop now:

> *In the code editor, write a program that counts down from 5 to 1 and then prints `"Liftoff!"`.*

Step-by-step plan (follow this structure):

1. Start with `count = 5`
2. `while` the count is at least 1
3. Inside the loop:
   - print the current count
   - then decrease it by 1
4. After the loop finishes, print `Liftoff!`

Do this in the code editor, run it to check, then hit Submit code.

I'll review it and then we'll do one last quick question about infinite loops ✅

```python
count = 5
while (count > 0):
    print(count)
    count = count - 1
print("Liftoff!")
```

That's excellent, Ron ?

```python
count = 5
while (count > 0):
    print(count)
    count = count - 1
print("Liftoff!")
```

You've nailed:

- Initialize: `count = 5`
- Test: `while (count > 0):`
- Update: `count = count - 1` so the loop must end
- Then a final message after the loop: `Liftoff!`

This is exactly how to safely avoid an infinite loop ✅

One last thing to lock this in:

In a `while` loop, which line is most responsible for avoiding an infinite loop?

I'll put this as a quick multiple-choice question next so you can choose an answer ?

the last line i n the loopa indented scope

You're thinking in exactly the right area: inside the indented loop body ?

More precisely, the key is the line that updates the variable used in the condition.

In your countdown:

```python
while (count > 0):
    print(count)
    count = count - 1   # ← this update saves you from an infinite loop
```

If we removed `count = count - 1`, `count` would stay 5 forever, `count > 0` would always be True, and the loop would never end.

So among typical lines:

- `print(...)` → just shows something
- `while (count > 0):` → checks the condition
- `count = count - 1` → actually changes things so the condition can become False ✅

Want to see this as a quick multiple-choice question to lock it in, or jump straight to a small "repeat-until-correct" style `while` loop (like a password checker)?

sure

What do you think? Pick the answer that feels most right ?

In a `while` loop like the one below, which line is most responsible for preventing an infinite loop?

```
count = 5
while count > 0:
    print(count)
    count = count - 1
print("Liftoff!")
```

A.

The `print(count)` line inside the loop

B.

The `while count > 0:` condition line

C.

The `count = count - 1` line inside the loop

D.

The `print("Liftoff!")` line after the loop

? Nice, Ron — you picked exactly the right line:

```
count = count - 1
```

That line is crucial because it changes the value of `count` every time the loop runs. Since the **while** condition is `count > 0`, reducing `count` each time guarantees that sooner or later the condition becomes False, so the loop stops.

- Without the update → `count` would stay the same → `count > 0` would stay True → infinite loop.
- With the update → `count` moves toward 0 → condition becomes False → loop ends ✅

You've got a solid grip on what a **while** loop is and how to avoid infinite loops.

Want to do one more practical **while** loop (like a simple password checker), or are you feeling ready to wrap up this lesson and move on?

Let's try the password loop

Nice choice ? Let's keep it simple.

Goal:

Keep asking for a password while it's wrong.

When it's `"opensesame"`, print a success message and stop.

The core idea in plain steps:

1. Ask for a password once before the loop.
2. Use a **while** loop that runs while the password is not correct.
3. Inside the loop:
   - tell the user it's wrong
   - ask again (update!)
4. After the loop, print something like `"Access granted"`.

You'll need **input()** for this.

Your turn in the code editor:

Write a program that:

- Repeatedly asks: `Enter the password:`
- Only stops when the user types exactly `opensesame`
- Then prints: **Access** granted

Run it to test (try a wrong password first), then hit Submit code and I'll check it ✅

Python

```
password = input("Enter a password: ")
while(password != "opesesame"):
    print("Im sorry that  is  incorrect")
    password = input("Please try again: ")
print("Access granted")
```

Nice work, Ron — you built the loop structure correctly ?

You've got:

- Initialize: first `input(...)`
- Test: `while(password != "opesesame")` (condition style is right)
- Update: new `input(...)` inside the loop
- After it ends → `print("Access granted")`

Two tiny fixes to make it behave as intended:

1. Spelling of the password

   You used `"opesesame"` in the code, but the requirement was `"opensesame"` (with an n).

   Right now, even if the user types `opensesame`, the loop will never stop.

2. Show what was incorrect (optional polish)

   Your message says `"Im sorry that is incorrect"` — there's a blank spot.

   If you wanted to show the wrong password, you could do:

```
print("I'm sorry, that", password, "is incorrect")
```

   (But this is just a bonus improvement, not required.)

   Can you fix the string in the `while` condition to `"opensesame"` in the code editor, run again (try a wrong password, then the correct one), and then Submit code one more time?

```
Python
password = input("Enter a password: ")
while(password != "opensesame"):
    print("Im sorry " + password + "is incorrect")
    password = input("Please try again: ")
print("Access granted")
```

   That's a great solution, Ron ?

```
Python
password = input("Enter a password: ")
while(password != "opensesame"):
    print("Im sorry " + password + "is incorrect")
    password = input("Please try again: ")
print("Access granted")
```

   Step by step, you've nailed it:

- Initialize: `password = input(...)`
- Condition: loop runs while `password != "opensesame"`
- Update inside loop: you ask again and store a new `password`
- Once the user types `"opensesame"`, the condition becomes False → loop stops → `"Access granted"` prints ✅

   This shows you understand:

- When a `while` loop is better than `for`
- How infinite loops happen and how to avoid them

- How to write a **while** loop that repeats until a condition changes

  You've basically mastered this lesson ?

  Do you feel ready to wrap up this while-loop lesson and move on to the next one, or would you like one more quick **while** practice first?

  I'm ready to move on

  Nice work today, Ron ?

  Here's what you can now do with confidence:
- Use a while loop when you don't know how many times to repeat, but you do know the condition (like "while password is wrong" or "while lives > 0").
- Spot and avoid infinite loops by always planning how the condition will eventually become **False**.
- Write working **while** loops that:
- initialize a variable,
- test a condition,
- update something inside the loop so it eventually stops.

  You built both a countdown and a password checker — great patterns to reuse later ?

  Whenever you're ready, you can jump into the next lesson from the Learn tab and keep building on this. If anything about **while** loops feels fuzzy later, just ask and we'll revisit it quickly.